Rapport de Projet Circuits et Architecture

VU Khanh Huyen, M1 IMPAIRS - PAN Alexis, M1 DATA

2025-2026

1 Introduction

Nous avons vu en cours comment fonctionnent les composants d'un micro-processeur LC-3 et également les bases pour coder des routines en assembleur. Le but de ce projet est de mettre en œuvre nos connaissances acquises en cours et d'implémenter dans un mini-processeur LC-3 des nouvelles opérations et des instructions standard, ainsi que de développer des routines en langage assembleur.

Plan

1 Introduction **1**

2 Explication du circuit LC-3 **1**
2.1 ALU . . . 1
2.2 DecodeIR . . . 2
2.3 NZP . . . 2
2.4 WriteVal . . . 2
2.5 RegPC . . . 2
2.6 Tests Circuit . . . 3

3 Code assembleur **3**
3.1 Manipulation de chaînes . . . 3
3.2 Opérations logiques . . . 3
3.3 Validation logicielle . . . 3

4 Conclusion **3**

2 Explication du circuit LC-3

Nous devions compléter les composants coloriés en rouge dans le circuit pour le minimum requis, ainsi que ceux en violet pour la partie avancée. De ce fait, nous avons dû modifier les modules **ALU**, **DecodeIR** et **NZP** pour implémenter les instructions **XOR** et **POPCNT**, puis **RegPC** et **WriteVal** pour implémenter les instructions standards **LEA, LDR, LD, ST, STR, BR, JMP, JSR et JSRR**.

2.1 ALU

- Nous avons fusionné le XOR et le NOT en les remplaçant par une porte **XOR**, étant donné que le LC-3 impose que les 5 derniers bits soient à 1 (valeur immédiate -1 en complément à 2). Ces instructions ont le même opcode (1001) et x XOR -1 équivaut à NOT x.
- Nous avons implémenté **POPCNT** en utilisant une structure d'arbre : on fait la somme de tous les bits par paires, puis la somme des résultats, afin d'obtenir le nombre final.
- Ainsi, dans le MUX de sortie, on a le choix parmi ADD, AND, XOR/NOT et POPCNT en fonction de l'opcode (signal e2).

2.2 DecodeIR

A la place de décoder les 16 opcodes individuellement, nous avons remarqué que les 2 bits de poids faible de l'opcode (bits 13 et 12 des instructions) permettent de séparer les instructions et d'activer les bons signaux.

- **Jump(00) :** Instructions de contrôle de flux (BR, JMP, JSR). Active le signal `Jump`.
- **Arith(01) :** Instructions arithmétiques et logiques (ADD, AND, XOR, POPCNT). Active le signal `Arith`.
- **Load(10) :** Instructions de chargement depuis la mémoire (LD, LDR, LDI, LEA). Active le signal `Load`.
- **Store(11) :** Instructions de stockage en mémoire (ST, STR, STI). Active le signal `Store`.
- **WriteReg** (écriture dans un registre) est généré via une porte OR si l'instruction est de type `Arith`, `Load` ou s'il s'agit spécifiquement de `LEA`. Cela garantit que le résultat est bien sauvegardé dans le banc de registres.

2.3 NZP

Le module NZP, qui contrôle les drapeaux N, Z et P pour les sauts conditionnels, se décompose ici en deux parties :

Mise à jour des indicateurs (Flags) : Le circuit analyse le résultat (RES) de la dernière opération pour déterminer son signe.

- **N (Negative) :** correspond directement au bit de poids fort (MSB, bit 15) du résultat.
- **Z (Zero) :** est activé si le résultat est égal à 0 (vérifié par un comparateur).
- **P (Positive) :** est déduit logiquement : si le nombre n'est ni Négatif ni Zéro.

Ces trois valeurs sont stockées dans un registre 3 bits. Ce registre ne se met à jour que si le signal `WriteReg` est actif.

Logique de saut (Branch Logic) : Pour l'instruction de branchement (BR), le circuit compare les indicateurs stockés avec les conditions demandées par l'instruction (bits 11 à 9 de l'IR). Si au moins une de ces conditions est remplie, le signal `TestNZP` passe à 1, autorisant le saut.

2.4 WriteVal

Le module WriteVal a pour rôle de sélectionner la donnée qui sera écrite dans le registre de destination (DR ou R7). Nous avons implémenté à l'aide de trois conditions (multiplexeur) :

- **Gestion du JSR :** Le premier multiplexeur vérifie si l'instruction est un JSR. Si oui, il sélectionne la valeur du `PC` pour la sauvegarder dans R7.
- **Gestion du LEA :** Le second vérifie si l'instruction est un LEA. Si oui, il sélectionne l'adresse calculée : $PC + SEXT(offset)$.
- **Chargement ou Calcul :** Le troisieme sélectionne soit une donnée lue en mémoire (`MemOUT`) si le signal `Load` est actif, soit le résultat de l'ALU (`ALURes`).

2.5 RegPC

L'objectif de ce module est de déterminer à chaque cycle quelle sera l'adresse de la prochaine instruction à exécuter.

Avant modification : Le circuit initial était conçu pour une exécution principalement linéaire. Il se contentait majoritairement d'incrémenter le compteur actuel ($PC + 1$), sans mécanisme flexible pour gérer les ruptures de séquence complexes (sauts).

Après modification : Pour supporter l'ensemble des instructions de contrôle de flux (**BR**, **JMP**, **JSR**, **JSRR**), nous avons dû introduire une logique de sélection en amont du registre PC. La modification majeure est l'ajout (ou l'activation) du multiplexeur **PCMux** qui permet désormais de choisir la prochaine valeur du PC parmi trois sources distinctes :

- **PC + 1 :** L'exécution séquentielle classique.
- **Sortie de l'ALU/Adder :** Pour les sauts relatifs (`BR`, `JSR`) où la nouvelle adresse est calculée en ajoutant un offset au PC actuel.
- **Bus de données :** Pour les sauts absolus (`JMP`, `JSRR`) où l'adresse de destination provient directement du contenu d'un registre (Base Register).

Cette architecture permet au processeur de "décider" dynamiquement de sauter ou de continuer tout droit selon l'instruction décodée.

2.6 Tests Circuit

Nous avons crées rois fichiers de tests unitaires qui nous assurent que tout se passe

- `test_alu` : Valide l'exactitude des calculs standards et confirme que les opcodes spécifiques 1001 (XOR) et 1101 (POPCNT) activent correctement les nouveaux chemins de données dans l'ALU.
- `test_mem` : Vérifie les accès mémoire en lecture/écriture ainsi que les modes d'adressage (PC-relatif vs Base+Offset), validant ainsi le basculement des multiplexeurs du module WriteVal.
- `test_flow` : Confirme la logique de saut (module NZP) et la sauvegarde correcte du PC dans R7 lors des appels de sous-routines (JSR/JSRR), assurant l'intégrité du séquencement.

3 Code assembleur

Dans cette partie, nous détaillons les routines logicielles implémentées pour exploiter l'architecture.

3.1 Manipulation de chaînes

index : Cette routine localise la première occurrence d'un caractère (R2) dans une chaîne (R1). Pour optimiser, nous utilisons l'arithmétique : tester $A = B$ revient à vérifier si $A + (-B) = 0$. Si le résultat est nul (`BRz`), l'occurrence est trouvée.

strcpy : Copie une chaîne source (R1) vers destination (R2). L'ordre des instructions (LDR puis STR puis Test 0) est crucial pour garantir que le caractère de fin de chaîne (zéro) est bien copié avant l'arrêt de la boucle.

strncpy : Cette variante sécurisée limite la copie à R0 caractères. La boucle s'interrompt si le caractère nul est rencontré ou si le compteur R0 atteint zéro.

3.2 Opérations logiques

reverse : Inverse les bits d'un mot de 16 bits (miroir). Une boucle de 16 itérations extrait le bit de poids fort (MSB) de l'entrée pour l'insérer en poids faible (LSB) dans le résultat par décalages successifs.

clz (Count Leading Zeros) : Compte les zéros en tête. Tant que le nombre est positif (bit de poids fort à 0), on incrémente le compteur et on décale à gauche. La boucle s'arrête dès que le bit de signe passe à 1.

ctz (Count Trailing Zeros) : Utilise les nouvelles instructions matérielles pour une exécution en $O(1)$ via la formule : $\text{CTZ}(x) = \text{POPCNT}((x-1) \oplus x) - 1$. L'opération $(x-1) \oplus x$ isole les zéros de fin, rendant le comptage immédiat.

3.3 Validation logicielle

Nous avons adopté une démarche de validation systématique : chaque routine développée est associée à un programme de test dédié. Ces tests ont été concus pour prouver que le circuit marche bien

4 Conclusion

La modification de l'architecture LC-3 nous a permis de mieux comprendre le fonctionnement du circuit et le développement des routines en assembleur nous a permit d'être plus a l'aise sur le langage assembleur.